

No- B_k CKKS Bootstrapping 分解中的实现归一化因子定位报告

实验整理与分析

2026-04-29

摘要

本文整理了对 no- B_k CKKS bootstrapping 分解中全局归一化因子 γ_{impl} 的定位过程。实验表明,理想代数层面的分解并不要求固定除以 k ;实际是否需要除以 k 取决于 Lattigo bootstrapping 参数中的 effective C2S/S2C scaling convention。对安全版本参数 $(\log N, k, \log n) = (17, 2, 16)$, top 和 leaf 的 effective C2S/S2C scaling product 相同,因此 $\gamma_{\text{impl}} = 1$, factored RAW 输出直接与 direct 输出对齐。对较小参数 $(\log N, k, \log n) = (16, 2, 15)$, leaf 的 effective SlotsToCoeffs scaling 比 top 大 2 倍,因此 $\gamma_{\text{impl}} = 2$, 需要 DIV-2 才能与 direct 对齐。根因链条为

$\text{LogMessageRatio} \rightarrow \text{MessageRatio} \rightarrow \text{StCScaling} \rightarrow \text{SlotsToCoeffsParameters.Scaling} \rightarrow \gamma_{\text{impl}}$.

这说明 RAW/DIV- k 差异不是 no- B_k 交换图的数学错误,而是实现层面的 normalization convention。

目录

1 背景与目标	2
2 理想数学交换图	2
2.1 Split 与 CoeffsToSlots 的交换	2
2.2 EvalMod 与 coordinate grouping 的交换	2
2.3 Merge 与 SlotsToCoeffs 的交换	3
3 实现中的归一化因子	3
4 诊断程序设计	3
4.1 核心思想	3
4.2 诊断阶段	4
4.3 关键 Go 诊断代码片段	4
5 关键实验数据	5
5.1 实验总表	5
5.2 $\log N = 17, k = 2$ 安全版参数	6
5.3 $\log N = 16, k = 2$ 对比参数	7

目录	2
6 根因定位: LogMessageRatio 到 S2C Scaling	8
6.1 Lattigo 参数中的可疑代码	8
6.2 Evaluator 中的 StCScaling 公式	8
6.3 两组参数的触发条件	9
7 为什么这不是数学错误	9
7.1 Split/Merge 单独没有引入因子	9
7.2 C2S 没有引入因子	10
7.3 EvalMod 没有引入因子	10
7.4 因子第一次出现在 S2C+Merge	10
8 性能与复杂度解释	10
8.1 直接路径与分解路径复杂度模型	10
8.2 实验时间	11
9 安全性备注	11
10 最终结论	11
A 附录: 建议报告用命令	12
B 附录: 关键源码逻辑摘要	13

1 背景与目标

本实验研究的是 CKKS full bootstrapping 的一种 no- B_k coefficient-split 分解。直接路径在大环 R_N 上执行完整 bootstrapping；分解路径先将密文拆成 k 个小环 R_n 密文，其中 $n = N/k$ ，然后对每个 leaf 执行小环 bootstrapping，最后 Merge 回大环。

直接 bootstrapping core 可抽象为

$$\text{BTS}_N = \text{S2C}_N \circ \text{EvalMod}_N \circ \text{C2S}_N. \quad (1)$$

factored no- B_k core 可抽象为

$$\text{BTS}_{N,k}^{\text{fac}} = \text{Merge}_k \circ \left(\bigoplus_{r=0}^{k-1} (\text{S2C}_n \circ \text{EvalMod}_n \circ \text{C2S}_n) \right) \circ \text{Split}_k. \quad (2)$$

完整 bootstrapping 则额外包括前处理：

$$\text{BTS}_N^{\text{full}} = \text{S2C}_N \circ \text{EvalMod}_N \circ \text{C2S}_N \circ \text{ModUp}_N \circ \text{ScaleDown}_N. \quad (3)$$

实验的最初目标是验证：

$$\text{BTS}_{N,k}^{\text{fac}} \approx \text{BTS}_N. \quad (4)$$

但是在不同参数下观察到两种现象：有些日志中 factored RAW 输出约为 direct 的 k 倍，需要 DIV- k ；另一些日志中 factored RAW 已经直接对齐 direct，不应除以 k 。本文目标就是定位这个差异来自哪里。

2 理想数学交换图

2.1 Split 与 CoeffsToSlots 的交换

令 S 表示 top C2S 输出上的 blocked coefficient grouping，即将大环坐标按 leaf block 重排。理想情况下应有

$$\left(\bigoplus_{r=0}^{k-1} \text{C2S}_n \right) \circ \text{Split}_k = S \circ \text{C2S}_N. \quad (5)$$

实验中通过 “C2S input to EvalMod: top vs Split+leaf layout comparison” 检查该关系。正确 layout 是 blocked layout，而不是 residue layout。所有成功实验中 blocked layout 的误差均在 10^{-11} 到 10^{-12} 量级。

2.2 EvalMod 与 coordinate grouping 的交换

EvalMod 是非线性的，但它是逐坐标作用的非线性。令

$$\text{EvalMod}_N = f^{\oplus N}, \quad \text{EvalMod}_n = f^{\oplus n}. \quad (6)$$

只要 S 是坐标重排或分组，便有

$$\left(\bigoplus_{r=0}^{k-1} f^{\oplus n} \right) \circ S = S \circ f^{\oplus N}. \quad (7)$$

因此这里不要求 f 线性；只要求 f 逐坐标作用。这解释了为什么虽然“非线性变换不能一般地穿过线性变换”，但 EvalMod 仍可穿过这种纯坐标分组结构。

2.3 Merge 与 SlotsToCoeffs 的交换

理想情况下也应有

$$\text{Merge}_k \circ \left(\bigoplus_{r=0}^{k-1} \text{S2C}_n \right) \circ S = \text{S2C}_N. \quad (8)$$

三步合成得到理想 no- B_k core 等式：

$$\text{Merge}_k \circ \left(\bigoplus_{r=0}^{k-1} (\text{S2C}_n \circ \text{EvalMod}_n \circ \text{C2S}_n) \right) \circ \text{Split}_k = \text{S2C}_N \circ \text{EvalMod}_N \circ \text{C2S}_N. \quad (9)$$

3 实现中的归一化因子

实际实现中需要写成

$$\text{BTS}_{N,k}^{\text{fac,impl}} \approx \gamma_{\text{impl}} \cdot \text{BTS}_N^{\text{impl}}. \quad (10)$$

其中 γ_{impl} 不是数学定理中的固定常数，而是由 Lattigo 内部 DFT/C2S/S2C scaling convention、bootstrapping parameter construction、以及 Split/Merge convention 决定。

根据 trace 实验，Split/Merge 本身、C2S、EvalMod 均没有引入全局倍数。真正差异第一次出现在 leaf S2C+Merge 相对 top S2C 的组合中。因此在当前程序中可用如下经验公式描述：

$$\gamma_{\text{impl}} = \frac{\text{CtwoSScaling}_{\text{leaf}} \cdot \text{S2CScaling}_{\text{leaf}}}{\text{CtwoSScaling}_{\text{top}} \cdot \text{S2CScaling}_{\text{top}}}. \quad (11)$$

更一般地，如果 Split/Merge API 也引入 normalization，则还应额外乘以 Split/Merge 的 normalization ratio。但本实验的 Split/Merge roundtrip trace 显示该因子为 1。

4 诊断程序设计

4.1 核心思想

诊断程序不只比较最终输出，而是逐阶段比较“want”与“got”，并拟合一个复比例 $\hat{\gamma}$ ：

$$\hat{\gamma} = \frac{\langle y_{\text{got}}, y_{\text{want}} \rangle}{\langle y_{\text{want}}, y_{\text{want}} \rangle}. \quad (12)$$

这里内积按复向量定义为

$$\langle a, b \rangle = \sum_i a_i \bar{b}_i. \quad (13)$$

如果

$$y_{\text{got}} \approx \gamma y_{\text{want}}, \quad (14)$$

则拟合得到的 $\hat{\gamma}$ 应接近真实全局因子。

每一阶段同时输出三类误差：

- RAW: 直接比较 y_{got} 与 y_{want} ;
- DIV- k : 比较 y_{got}/k 与 y_{want} ;
- FIT-GAMMA: 比较 $y_{\text{got}}/\hat{\gamma}$ 与 y_{want} 。

4.2 诊断阶段

诊断程序记录以下阶段:

1. ScaleDown: direct vs factored;
2. ModUp: direct vs factored;
3. Split+Merge on ModUp;
4. C2S real: top vs blocked leaves;
5. C2S imag: top vs blocked leaves;
6. Top C2S→S2C vs ModUp;
7. Leaf C2S→S2C+Merge vs ModUp;
8. Leaf linear path vs top linear path;
9. EvalMod real: top vs blocked leaves;
10. EvalMod imag: top vs blocked leaves;
11. Final S2C+Merge after EvalMod.

其中第 6-8 项是额外的 linear-only S2C diagnostics, 不经过 EvalMod, 用于隔离 S2C/Merge normalization。

4.3 关键 Go 诊断代码片段

下面是 DFT 参数 dump 函数, 用于打印 top/leaf C2S/S2C 的 effective MatrixLiteral 参数。

Listing 1: DFT MatrixLiteral dump 函数

```
func dumpDFTParams(name string, p dft.MatrixLiteral) {
    fmt.Printf("\n--- %s ---\n", name)
    fmt.Printf("Type           = %v\n", p.Type)
    fmt.Printf("LogSlots         = %d\n", p.LogSlots)
    fmt.Printf("Slots            = %d\n", 1<<p.LogSlots)
    fmt.Printf("2*Slots          = %d\n", 2*(1<<p.LogSlots))
    fmt.Printf("1/Slots          = %.17e\n", 1.0/float64(1<<p.LogSlots))
    fmt.Printf("1/(2Slots)       = %.17e\n", 1.0/float64(2*(1<<p.LogSlots)))
    fmt.Printf("Format           = %v\n", p.Format)
```

```

    fmt.Printf("BitReversed = %v\n", p.BitReversed)
    fmt.Printf("LevelQ      = %d\n", p.LevelQ)
    fmt.Printf("LevelP      = %d\n", p.LevelP)
    fmt.Printf("Levels      = %v\n", p.Levels)
    fmt.Printf("Depth(false)= %d\n", p.Depth(false))
    fmt.Printf("Depth(true) = %d\n", p.Depth(true))

    if p.Scaling == nil {
        fmt.Printf("Scaling      = nil (default 1.0 before Lattigo internal
                    scaling)\n")
    } else {
        sf, _ := p.Scaling.Float64()
        fmt.Printf("Scaling      = %s\n", p.Scaling.Text('g', 18))
        fmt.Printf("Scaling f64 = %.17e\n", sf)
        fmt.Printf("Scaling/Slots    = %.17e\n", sf/float64(1<<p.LogSlots))
        fmt.Printf("Scaling/(2Slots) = %.17e\n", sf/float64(2*(1<<p.
                    LogSlots)))
    }
}

```

插入位置为 top evaluator 与 leaf evaluator pool 创建完成后:

Listing 2: 在 main 中打印 top/leaf DFT 参数

```

fmt.Println()
fmt.Println("=== DFT MatrixLiteral dump: TOP vs LEAF ===")

dumpDFTParams("TOP effective CoeffsToSlots", topEval.
    CoeffsToSlotsParameters)
dumpDFTParams("TOP effective SlotsToCoeffs", topEval.
    SlotsToCoeffsParameters)

dumpDFTParams("LEAF[0] effective CoeffsToSlots", leafEvals[0].
    CoeffsToSlotsParameters)
dumpDFTParams("LEAF[0] effective SlotsToCoeffs", leafEvals[0].
    SlotsToCoeffsParameters)

```

5 关键实验数据

5.1 实验总表

Case	top log N	k	leaf log n	normalization	final check	levels	speedup
$N13-k2$	13	2	12	DIV-2	7.91×10^{-8}	10/10	$1.83\times$
$N16-k4$	16	4	14	DIV-4	1.09×10^{-7}	10/10	$2.49\times$

$N16-k2$	16	2	15	DIV-2	2.41×10^{-7}	10/10	$3.64 \times$
$N17-k2$	17	2	16	RAW	$6.5 \sim 7.7 \times 10^{-7}$	10/10	$5.03 \times$

这里 final check 表示 implementation-normalized direct-vs-factored 最大误差。对 DIV- k case, 使用 factored/ k ; 对 RAW case, 直接使用 factored RAW。

5.2 $\log N = 17, k = 2$ 安全版参数

该组是本文最重要的安全-aware 实验:

$$\log N_{\text{top}} = 17, \quad k = 2, \quad \log n_{\text{leaf}} = 16. \quad (15)$$

基本参数:

- top LogN = 17, $N = 131072$;
- top logSlots = 16, slots = 65536;
- leaf LogN = 16, $n = 65536$;
- leaf logSlots = 15, slots = 32768;
- $\log(QP) = 1520.0$;
- top/leaf Q/P basis 完全一致;
- output level: direct = 10, factored = 10.

该组 DFT dump 给出:

表 2: $\log N = 17, k = 2$ 的 effective DFT scaling

Component	LogSlots	C2S Scaling	S2C Scaling
Top	16	$0.0625 \approx 2^{-4}$	$0.000244140625 = 2^{-12}$
Leaf	15	$0.0625 \approx 2^{-4}$	$0.000244140625 = 2^{-12}$

因此:

$$\text{CtwoSScaling}_{\text{top}} \text{S2CScaling}_{\text{top}} = 2^{-4} \cdot 2^{-12} = 2^{-16}, \quad (16)$$

$$\text{CtwoSScaling}_{\text{leaf}} \text{S2CScaling}_{\text{leaf}} = 2^{-4} \cdot 2^{-12} = 2^{-16}. \quad (17)$$

所以

$$\boxed{\gamma_{\text{impl}} = 1.} \quad (18)$$

Normalization trace 也直接验证:

Listing 3: $\log N = 17, k = 2$ normalization trace 摘要

06 Top C2S->S2C vs ModUp	gamma=+1.52587891e-05
07 Leaf C2S->S2C+Merge vs ModUp	gamma=+1.52587891e-05
08 Leaf linear path vs top path	gamma=+1.00000000e+00
11 Final S2C+Merge after EvalMod	gamma=+1.00000000e+00

最终质量:

- Direct vs plaintext max/RMSE: $6.94 \times 10^{-8} / 2.09 \times 10^{-8}$ 或同量级;
- Factored RAW vs plaintext max/RMSE: $6.55 \times 10^{-7} / 1.98 \times 10^{-7}$ 或同量级;
- Direct vs factored RAW max/RMSE: $6.5 \sim 7.7 \times 10^{-7} / 2.0 \times 10^{-7}$;
- Direct vs factored DIV-2 max/RMSE: $5.238 \times 10^{-1} / 3.217 \times 10^{-1}$, 明显错误。

因此该组结论为:

$$\text{Merge}_2 \circ (\text{BTS}_{16} \oplus \text{BTS}_{16}) \circ \text{Split}_2 \approx \text{BTS}_{17} \quad \text{in RAW convention.} \quad (19)$$

5.3 $\log N = 16, k = 2$ 对比参数

该组为:

$$\log N_{\text{top}} = 16, \quad k = 2, \quad \log n_{\text{leaf}} = 15. \quad (20)$$

DFT dump 给出:

表 3: $\log N = 16, k = 2$ 的 effective DFT scaling

Component	LogSlots	C2S Scaling	S2C Scaling
Top	15	$0.0625 \approx 2^{-4}$	$0.000244140625 = 2^{-12}$
Leaf	14	$0.0625 \approx 2^{-4}$	$0.00048828125 = 2^{-11}$

因此:

$$\text{CtwoSScaling}_{\text{top}} \S 2CScaling_{\text{top}} = 2^{-4} \cdot 2^{-12} = 2^{-16}, \quad (21)$$

$$\text{CtwoSScaling}_{\text{leaf}} \S 2CScaling_{\text{leaf}} = 2^{-4} \cdot 2^{-11} = 2^{-15}. \quad (22)$$

于是

$$\gamma_{\text{impl}} = \frac{2^{-15}}{2^{-16}} = 2. \quad (23)$$

Normalization trace 验证:

Listing 4: $\log N = 16, k = 2$ normalization trace 摘要

06 Top C2S->S2C vs ModUp	gamma=+1.52587891e-05
07 Leaf C2S->S2C+Merge vs ModUp	gamma=+3.05175781e-05
08 Leaf linear path vs top path	gamma=+2.00000000e+00
11 Final S2C+Merge after EvalMod	gamma=+2.00000000e+00

最终质量:

- Direct vs plaintext max/RMSE: $3.18 \times 10^{-8} / 1.04 \times 10^{-8}$;
- Factored RAW vs plaintext max/RMSE: $1.047691 / 0.643471$, 显示整体约 2 倍错误;
- Factored DIV-2 vs plaintext max/RMSE: $2.43 \times 10^{-7} / 6.98 \times 10^{-8}$;
- Direct vs factored DIV-2 max/RMSE: $2.41 \times 10^{-7} / 7.05 \times 10^{-8}$.

因此该组结论为:

$$\frac{1}{2} \text{Merge}_2 \circ (\text{BTS}_{15} \oplus \text{BTS}_{15}) \circ \text{Split}_2 \approx \text{BTS}_{16}. \quad (24)$$

6 根因定位: LogMessageRatio 到 S2C Scaling

6.1 Lattigo 参数中的可疑代码

grep 发现 Lattigo 测试与示例中存在针对小 LogSlots 或小 LogN 的 LogMessageRatio 补偿, 例如:

Listing 5: LogMessageRatio 小参数补偿示例

```
btpParams.Mod1ParametersLiteral.LogMessageRatio +=
    utils.Min(utils.Max(15-LogSlots, 0), 8)
```

以及:

Listing 6: 按 LogN 的 MessageRatio 补偿示例

```
btpParams.Mod1ParametersLiteral.LogMessageRatio += 16 - params.LogN()
```

这些代码未必直接在 library runtime 中自动执行, 但实验程序构造参数时采用了类似 convention; 最终 DFT dump 证明实际 evaluator 已经接收到了补偿后的 effective scaling。

6.2 Evaluator 中的 StCScaling 公式

bootstrapping evaluator 中构造 S2C scaling 的关键公式为:

Listing 7: StCScaling 构造逻辑

```
scale := eval.BootstrappingParameters.DefaultScale().Float64()
offset := eval.Mod1Parameters.ScalingFactor().Float64() /
    eval.Mod1Parameters.MessageRatio()

StCScaling := new(big.Float).SetFloat64(scale / offset)

if btpParams.SlotsToCoeffsParameters.Scaling == nil {
    eval.SlotsToCoeffsParameters.Scaling = StCScaling
} else {
```

```
eval.SlotsToCoeffsParameters.Scaling =
    new(big.Float).Mul(btpParams.SlotsToCoeffsParameters.Scaling,
        StCScaling)
}
```

数学上:

$$\text{offset} = \frac{\text{ScalingFactor}}{\text{MessageRatio}}, \quad (25)$$

所以

$$\text{StCScaling} = \frac{\text{scale}}{\text{offset}} = \frac{\text{scale} \cdot \text{MessageRatio}}{\text{ScalingFactor}}. \quad (26)$$

又因为

$$\text{MessageRatio} = 2^{\text{LogMessageRatio}}, \quad (27)$$

所以如果

$$\text{LogMessageRatio} \leftarrow \text{LogMessageRatio} + 1, \quad (28)$$

则

$$\text{MessageRatio} \leftarrow 2\text{MessageRatio}, \quad \text{StCScaling} \leftarrow 2\text{StCScaling}. \quad (29)$$

这正好解释了 leaf S2C scaling 从 2^{-12} 变为 2^{-11} 的现象。

6.3 两组参数的触发条件

若使用

$$\Delta_{\text{LMR}} = \max(15 - \text{LogSlots}, 0) \quad (30)$$

这类补偿, 则:

表 4: LogMessageRatio 补偿触发情况

Case	leaf LogN	leaf LogSlots	$\max(15 - \text{LogSlots}, 0)$
$\log N = 17, k = 2$	16	15	0
$\log N = 16, k = 2$	15	14	1

所以:

- $\log N = 17, k = 2$: leaf LogSlots=15, 不触发补偿, $\gamma_{\text{impl}} = 1$;
- $\log N = 16, k = 2$: leaf LogSlots=14, 触发 +1 补偿, leaf S2C scaling $\times 2$, 所以 $\gamma_{\text{impl}} = 2$.

7 为什么这不是数学错误

7.1 Split/Merge 单独没有引入因子

在两组 normtrace 中, Split+Merge on ModUp 均显示

$$\gamma \approx 1. \quad (31)$$

因此 Merge API 单独不是全局因子的来源。

7.2 C2S 没有引入因子

C2S real/imag 的 blocked layout 比较均显示

$$\gamma \approx 1, \quad \text{error} \approx 10^{-11}. \quad (32)$$

这说明

$$(\oplus \text{C2S}_n) \circ \text{Split}_k \quad (33)$$

与

$$S \circ \text{C2S}_N \quad (34)$$

在 blocked layout 下正确对齐。

7.3 EvalMod 没有引入因子

EvalMod real/imag 比较均显示

$$\gamma \approx 1, \quad (35)$$

误差在 10^{-5} 到 10^{-6} 量级, 符合 EvalMod 近似误差特征。因此全局因子不是 EvalMod 的非线性导致的。

7.4 因子第一次出现在 S2C+Merge

对 $\log N = 16, k = 2$, 第 7-8 阶段显示 leaf linear path 相对 top linear path 的 fitted gamma 为 2; 第 11 阶段 final S2C+Merge after EvalMod 也为 2。这说明因子首次出现在 leaf S2C+Merge 相对 top S2C 的归一化差异中。

对 $\log N = 17, k = 2$, 第 7-8 阶段与第 11 阶段均显示 gamma 为 1, 因此 RAW 对齐。

8 性能与复杂度解释

8.1 直接路径与分解路径复杂度模型

直接路径:

$$T_{\text{dir}}(N) = T_{\text{ScaleDown}}(N) + T_{\text{ModUp}}(N) + T_{\text{C2S}}(N) + T_{\text{EvalMod}}(N) + T_{\text{S2C}}(N). \quad (36)$$

分解路径:

$$T_{\text{fac}}(N, k) = T_{\text{ScaleDown}}(N) + T_{\text{ModUp}}(N) + T_{\text{Split}}(N, k) + T_{\text{leaf-par}}(N/k) + T_{\text{Merge}}(N, k), \quad (37)$$

其中并行 leaf 部分近似为

$$T_{\text{leaf-par}}(n) \approx \max_{0 \leq r < k} \left(T_{\text{C2S}}^{(r)}(n) + T_{\text{EvalMod}}^{(r)}(n) + T_{\text{S2C}}^{(r)}(n) \right) + T_{\text{sync}}. \quad (38)$$

若 bootstrapping core 的实际增长近似

$$T_{\text{core}}(N) \approx cN^p, \quad (39)$$

且 $p > 1$, 则理论 speedup 可以超过 k :

$$\text{Speedup} \approx \frac{cN^p}{c(N/k)^p + T_{\text{overhead}}}. \quad (40)$$

8.2 实验时间

表 5: Full bootstrapping timing

Case	Direct full	Factored full	Speedup
$\log N = 13, k = 2$	0.723888s	0.395569s	$1.8300\times$
$\log N = 16, k = 4$	14.362138s	5.761283s	$2.4929\times$
$\log N = 16, k = 2$	26.282260s	7.220363s	$3.6400\times$
$\log N = 17, k = 2$	117.207541s	23.303824s	$5.0295\times$

$\log N = 17, k = 2$ 的 speedup 超过 $2\times$ ，说明收益不只是两个 leaf 并行，而是大环 direct bootstrapping 的 C2S/S2C/key switching/NTT/memory movement 成本具有明显超线性增长。分解路径把一个超线性大任务替换为两个较小、cache 更友好的 leaf bootstrapping，并行执行后得到显著加速。

9 安全性备注

本报告主要聚焦 normalization 定位。但从参数安全角度， $\log N = 17, k = 2$ 有额外意义：它的 leaf 维度为 $\log n = 16$ ，可承载 bootstrapping 级别的 $\log(QP) \approx 1520$ ，比 earlier prototype 中 leaf $\log n = 14$ 或 15 更接近常见 CKKS bootstrapping 安全参数范围。

正式安全结论仍应以 lattice/RLWE estimator 为准，并对以下 component 分别估计：

$$\lambda_{\text{scheme}} = \min\{\lambda_{\text{top}}, \lambda_{\text{leaf}}, \lambda_{\text{SplitKey}}, \lambda_{\text{MergeKey}}, \lambda_{\text{BTSKey}}\}. \quad (41)$$

本文不声称已经完成 estimator 级别的最终安全证明。

10 最终结论

本次定位已经较完整地解释了 RAW/DIV- k 差异：

1. 理想 $\text{no-}B_k$ 代数交换图成立，EvalMod 虽然非线性，但因其逐坐标作用，所以可以与 blocked coordinate grouping 交换。
2. 实现中存在全局 normalization factor γ_{impl} 。
3. γ_{impl} 不是固定等于 k ，也不是固定等于 1；它由 effective C2S/S2C scaling product 决定。
4. 对 $\log N = 17, k = 2, \log n = 16$ ，top 与 leaf scaling product 相同，故 $\gamma_{\text{impl}} = 1$ ，RAW 正确。
5. 对 $\log N = 16, k = 2, \log n = 15$ ，leaf S2C scaling 比 top 大 2 倍，故 $\gamma_{\text{impl}} = 2$ ，DIV-2 正确。

6. 该 S2C scaling 差异可由 $\text{LogMessageRatio} \rightarrow \text{MessageRatio} \rightarrow \text{StCScaling} \rightarrow \text{SlotsToCoeffsParameters.Scaling}$ 链条解释。当 leaf LogSlots 降到 14 时，LogMessageRatio 补偿增加 1，使 S2C scaling 乘以 2。
7. 因此，RAW/DIV- k 差异不是 Split、Merge、C2S 或 EvalMod 的数学错误，而是 Lattigo bootstrapping 参数 scaling convention 的实现现象。

最简洁的结论公式为：

$$\text{BTS}_{N,k}^{\text{fac,impl}} \approx \gamma_{\text{impl}} \text{BTS}_N^{\text{impl}}, \quad \gamma_{\text{impl}} = \frac{\text{CtwoSScaling}_{\text{leaf}}^{\text{S2CScaling}_{\text{leaf}}}}{\text{CtwoSScaling}_{\text{top}}^{\text{S2CScaling}_{\text{top}}}}. \quad (42)$$

对后续论文或汇报，建议写：

In the ideal algebraic model, the no- B_k decomposition does not require a fixed division by k . In the implementation, a global normalization factor γ_{impl} may appear due to Lattigo's bootstrapping scaling convention. We locate this factor to the effective SlotsToCoeffs scaling, which depends on MessageRatio/LogMessageRatio and hence on the chosen bootstrapping parameters. For the security-aware run $(\log N, k, \log n) = (17, 2, 16)$, we empirically observe $\gamma_{\text{impl}} = 1$, so the raw factored output directly matches the direct bootstrapping output.

A 附录：建议报告用命令

Listing 8: 安全版 $\log N = 17, k = 2$ 诊断命令

```
go run . \
-logN=17 \
-layers=1 \
-leafOrder=bitrev \
-parallelLeaves=true \
-leafWorkers=2 \
-reps=1 \
-warmup=0 \
-timing=false | tee dftparams_logN17_k2.txt
```

Listing 9: $\log N = 16, k = 2$ 对照诊断命令

```
go run . \
-logN=16 \
-layers=1 \
-leafOrder=bitrev \
-parallelLeaves=true \
-leafWorkers=2 \
-reps=1 \
-warmup=0 \
-timing=false | tee dftparams_logN16_k2.txt
```

Listing 10: $\log N = 17, k = 2$ timing 命令

```
go run . \
  -logN=17 \
  -layers=1 \
  -leafOrder=bitrev \
  -parallelLeaves=true \
  -leafWorkers=2 \
  -reps=1 \
  -warmup=0 \
  -timing=true | tee timing_logN17_k2.txt
```

B 附录：关键源码逻辑摘要

Listing 11: LogMessageRatio 补偿逻辑

```
// In tests/examples or parameter construction conventions:
btpParams.Mod1ParametersLiteral.LogMessageRatio +=
  utils.Min(utils.Max(15-LogSlots, 0), 8)
```

Listing 12: MessageRatio 进入 StCScaling

```
offset := eval.Mod1Parameters.ScalingFactor().Float64() /
  eval.Mod1Parameters.MessageRatio()

StCScaling := new(big.Float).SetFloat64(scale / offset)
```

由此可得：

$$\text{StCScaling} = \frac{\text{scale} \cdot \text{MessageRatio}}{\text{ScalingFactor}}. \quad (43)$$

因此 LogMessageRatio 增加 1 会导致 StCScaling 乘以 2。